

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

COMPUTACIÓN GRÁFICA E INTERACCIÓN HUMANO-COMPUTADORA

SEMESTRE 2027 - 1



MODELADO

ALUMNOS	*Basilio Illescas Andrés*	*Product Owner*
	Trejo Olvera Emmanuel	*Scrum Master*
	Pérez Olvera Alexis Abraham	*Developer*
	Cruz Macedo Samuel Santiago	*Developer*
PROFESOR	*Ing. José Ramón Pérez Athié*	
GRUPO	*04*	

Ciudad Universitaria, CDMX

13 de octubre de 2026

Índice

1. Introducción **2**

2. Objetivos **2**

2.1. Objetivo general 2

2.2. Objetivos específicos 2

3. Alcance de la entrega **2**

4. Metodología de trabajo ágil **3**

4.1. Roles 3

4.2. Aplicación de Scrum 3

5. Diseño y desarrollo **4**

5.1. Concepto espacial 4

5.2. Modelado geométrico 4

5.3. Distribución de modelos propios 4

5.4. Jerarquías e instancias 5

5.5. Visualización y navegación 5

6. Estructura del proyecto **5**

7. Pruebas realizadas **6**

8. Capturas de la aplicación **7**

9. Compilación y ejecución **8**

9.1. macOS con Xcode 8

9.2. Windows con Visual Studio 8

9.3. Linux 9

9.4. Controles 9

10. Resultados **9**

11. Conclusiones y trabajo posterior **9**

12. Referencias **10**

1. Introducción

El modelado geométrico constituye la base visual de una aplicación tridimensional. Antes de incorporar texturas, materiales avanzados, iluminación compleja o animaciones, es necesario definir correctamente las formas, proporciones, jerarquías y distribución espacial de los objetos. En esta etapa se desarrolló una escena inspirada en la celebración mexicana del Día de Muertos, cuyo elemento principal es un altar conectado mediante un camino de pétalos con la entrada de un panteón.

La aplicación fue programada en C++17 y utiliza OpenGL 3.3 para presentar los modelos en tiempo real. Los objetos propios se elaboraron en Blender y se exportaron en formato glTF 2.0. También se integró un conjunto de modelos de librería con licencia CC0 para complementar el entorno. La escena puede recorrerse con una cámara libre y cuenta con una visualización de alambre que permite observar la construcción poligonal.

Esta entrega se concentra exclusivamente en el **modelado** del proyecto final. Todos los objetos se presentan sin materiales o texturas ni colores, con una iluminación direccional por defecto.

2. Objetivos

2.1. Objetivo general

Construir e integrar una escena tridimensional funcional sobre la tradición del Día de Muertos, compuesta por modelos acorde a esta celebración, que permita evaluar su geometría, proporciones, jerarquías y distribución mediante renderizado sólido y de alambre en OpenGL.

2.2. Objetivos específicos

- Diseñar una composición que conecte un altar de día de muertos y un panteón.
- Crear al menos ocho modelos originales; en esta entrega se presentan dieciocho.
- Integrar modelos complementarios de una librería con licencia compatible.
- Conservar jerarquías para objetos formados por varias piezas.
- Reutilizar geometría mediante instancias cuando un elemento aparece muchas veces.
- Implementar una cámara sencilla y un modo de alambre.
- Mantener el proyecto compilable en macOS, Windows y Linux mediante CMake.

3. Alcance de la entrega

El alcance comprende el diseño geométrico, la carga de archivos glTF/GLB, la composición de la escena, el modelado jerárquico, el recorrido con cámara y la visualización sólida o de alambre. El programa requiere OpenGL 3.3 o superior.

Los objetos se muestran en escala de grises para concentrar la evaluación en siluetas y volúmenes. Los modelos que posteriormente emplearán transparencia o recortes —por ejemplo el papel picado

y las alas de la mariposa— muestran por ahora su superficie completa. El vaso y el agua también se presentan como objetos opacos.

Quedan fuera de esta entrega las texturas, materiales metálicos, emisión, transparencia, reflejos, iluminación dinámica, niebla, cielo panorámico y animaciones.

4. Metodología de trabajo ágil

Para organizar el trabajo se adoptó una variante ligera de *Scrum*, adecuada a la duración de una entrega académica. Se buscó mantener tareas visibles, incrementos pequeños y revisiones frecuentes.

4.1. Roles

Integrante	Rol	Responsabilidad
Basilio Illescas Andrés	Product Owner	Priorizar elementos y revisar que la escena represente el tema.
Trejo Olvera Emmanuel	Scrum Master y desarrollador	Coordinar avances, retirar bloqueos e integrar el sistema floral.
Pérez Olvera Alexis Abraham	Desarrollador	Modelar objetos de la ofrenda y verificar su integración.
Cruz Macedo Samuel Santiago	Desarrollador	Modelar elementos arquitectónicos y ambientales.

Tabla 1: Roles del equipo para la entrega de modelado.

4.2. Aplicación de Scrum

El *Product Backlog* se formó con historias breves. Un ejemplo fue: “Como visitante, quiero recorrer un altar y un panteón reconocibles para identificar los elementos principales del Día de Muertos”. Cada historia se dividió en tareas verificables: modelar, revisar escala, corregir origen, exportar, integrar y probar.

El trabajo se dividió en cuatro iteraciones:

1. **Planeación y escena base:** recorrido, unidades, orientación de ejes y lista de modelos.
2. **Producción:** elaboración individual y revisión cruzada de silueta, proporción y complejidad.
3. **Integración:** exportación glTF, colocación, jerarquías e instancias.
4. **Validación:** compilación, recorrido, revisión sólida y de alambre, y correcciones.

En reuniones breves cada integrante comunica qué terminó, qué hará después y qué impedimento encontró. Al cerrar una iteración se revisa el incremento y se realiza una retrospectiva. Git registra versiones mediante *commits* pequeños y puntos estables.

La definición de terminado para un modelo incluye: escala coherente, origen correcto, transformaciones aplicadas, normales orientadas, ausencia de geometría accidental, nombre identificable, exportación válida e integración comprobada en vista sólida y de alambre.

5. Diseño y desarrollo

5.1. Concepto espacial

En un extremo se ubica una casa de adobe con el altar de tres niveles y sus ofrendas. En el centro se extiende un camino de pétalos acompañado por veladoras, faroles y papel picado. En el extremo opuesto se encuentra el panteón, delimitado por bardas y rejas, con tumbas, criptas, vegetación y una entrada monumental.

Cada modelo tiene su origen cerca de la base para facilitar su colocación sobre el terreno. La escena agrupa piezas relacionadas en nodos padre e hijo, por lo que los componentes de un altar, farol o edificio conservan su relación espacial.

5.2. Modelado geométrico

Los modelos propios se construyeron en Blender mediante extrusión, escalado, rotación, superficies de revolución, deformación de vértices y duplicación controlada. Los elementos repetidos, como flores, pétalos, veladoras, bardas y vegetación, reutilizan geometría para evitar almacenar copias completas.

El formato glTF conserva mallas, índices, normales, jerarquías y transformaciones. En esta entrega se usa un acabado gris mate uniforme desde el shader. Los modelos de librería se conservan en formato GLB y sirven como apoyo ambiental.

5.3. Distribución de modelos propios

La Tabla 2 distribuye los dieciocho modelos originales entre los integrantes. La autoría corresponde al responsable principal; la revisión e integración se realizaron en equipo.

Tabla 2: Responsables de los modelos propios.

Integrante	Modelos	Función
Basilio Illescas Andrés	Altar, veladora, cirio con candelero y pan de muerto	Estructura principal y objetos rituales de los niveles.
Trejo Olvera Emmanuel	Calaverita, pétalo, flor y arco de cempasúchil; maceta de cempasúchil	Elementos distintivos de la ofrenda y sistema floral reutilizable.
Pérez Olvera Alexis Abraham	Papel picado, sahumerio, portarretrato, jarrón de Talavera y vaso con agua	Decoración, recipientes y ofrendas del altar.
Cruz Macedo Samuel Santiago	Mariposa monarca, farol, entrada del panteón y casa de adobe	Ambientación, arquitectura y delimitación del recorrido.

También se integraron veinticinco objetos de *Kenney Graveyard Kit*, con licencia CC0: lápidas, cruces, criptas, bardas, rejas, árboles, rocas, bancas y personajes. Complementan el contexto sin sustituir los modelos originales requeridos.

5.4. Jerarquías e instancias

La clase `Nodo` almacena una transformación local de traslación, rotación y escala. La transformación de mundo se obtiene multiplicando la matriz del padre por la matriz local del hijo. La clase `Lote` agrupa instancias de una misma malla; cada copia conserva su matriz, pero la geometría se almacena una sola vez. Esto se utiliza en pétalos, flores, veladoras, árboles y cercas.

5.5. Visualización y navegación

El shader de vértices transforma posiciones y normales. El shader de fragmentos aplica luz ambiental y una iluminación direccional difusa, sin reflejos ni propiedades metálicas. Con `F2`, OpenGL cambia al modo de líneas para mostrar las aristas.

La cámara utiliza `WASD` para desplazarse, `Q/E` para bajar o subir y el botón derecho para mirar. `Shift` aumenta la velocidad, la rueda ajusta la velocidad base y `R` regresa al punto inicial.

6. Estructura del proyecto

Las carpetas de compilación, como `build-xcode` o `build-windows`, se generan localmente y no forman parte del código fuente.

```
entrega-modelado/
├── CMakeLists.txt
├── README.md
├── readme.txt
├── LICENSE
├── assets/models/
│   ├── propios/
│   └── libreria/
├── doc/Equipo-8-modelado.tex
├── external/
│   ├── cgltf/ ├── glad/ ├── glfw/ └── glm/
├── shaders/
│   ├── escena.vert
│   └── escena.frag
└── src/
    ├── main.cpp
    ├── App.cpp / App.h
    ├── Rutas.cpp / Rutas.h
    ├── Registro.cpp / Registro.h
    ├── bibliotecas.cpp
    ├── render/
    └── scene/
```

Tabla 3: Archivos y carpetas importantes.

Elemento	Propósito
`CMakeLists.txt`	Define la versión de C++, fuentes, bibliotecas, opciones de plataforma y copia de recursos.
`src/main.cpp`	Punto de entrada; inicializa rutas y crea la aplicación.

Elemento	Propósito
src/App.*	Crea la ventana OpenGL, mantiene el ciclo principal y procesa teclado y ratón.
src/Rutas.*	Localiza assets y shaders durante desarrollo y distribución.
src/Registro.*	Guarda mensajes y errores para facilitar las pruebas.
src/render/Model.*	Lee glTF/GLB y crea buffers de vértices e índices.
src/render/Shader.*	Compila y enlaza los programas GLSL.
src/render/Renderer.*	Recorre nodos visibles y dibuja la escena sólida o de alambre.
src/scene/Camera.*	Implementa cámara libre y proyección en perspectiva.
src/scene/SceneNode.*	Implementa el grafo jerárquico y lotes de instancias.
src/scene/Camino.*	Define la curva para distribuir pétalos y objetos laterales.
src/scene/Escena.*	Carga modelos y construye ofrenda, camino, panteón y alrededores.
src/bibliotecas.cpp	Activa la implementación del lector glTF.
shaders/escena.vert	Transforma vértices y normales; admite dibujo instanciado.
shaders/escena.frag	Calcula el gris mate con iluminación sencilla.
assets/models/propios	Archivos glTF y buffers creados por el equipo.
assets/models/libreria	Modelos GLB de apoyo y licencia.
external	GLFW, GLAD, GLM y cgltf.
README.md	Alcance, controles, compilación y estructura para desarrolladores.
readme.txt	Instrucciones breves copiadas junto al ejecutable.

GLFW administra ventana y entrada; GLAD carga funciones OpenGL; GLM proporciona vectores, matrices y cuaterniones; y cgltf interpreta los archivos glTF/GLB. Estas cuatro dependencias se utilizan directamente.

7. Pruebas realizadas

- Compilación de depuración en macOS, Arch Linux, Ubuntu y Windows.
- Creación de ventana y contexto OpenGL.
- Carga de modelos y buffers.
- Recorrido mediante teclado y mouse.
- Regreso al inicio mediante R.
- Cambio entre vista sólida y alambre mediante F2.
- Revisión visual de escalas, posiciones, normales y geometría repetida.
- Comprobación de interacción fluida durante el recorrido.

8. Capturas de la aplicación

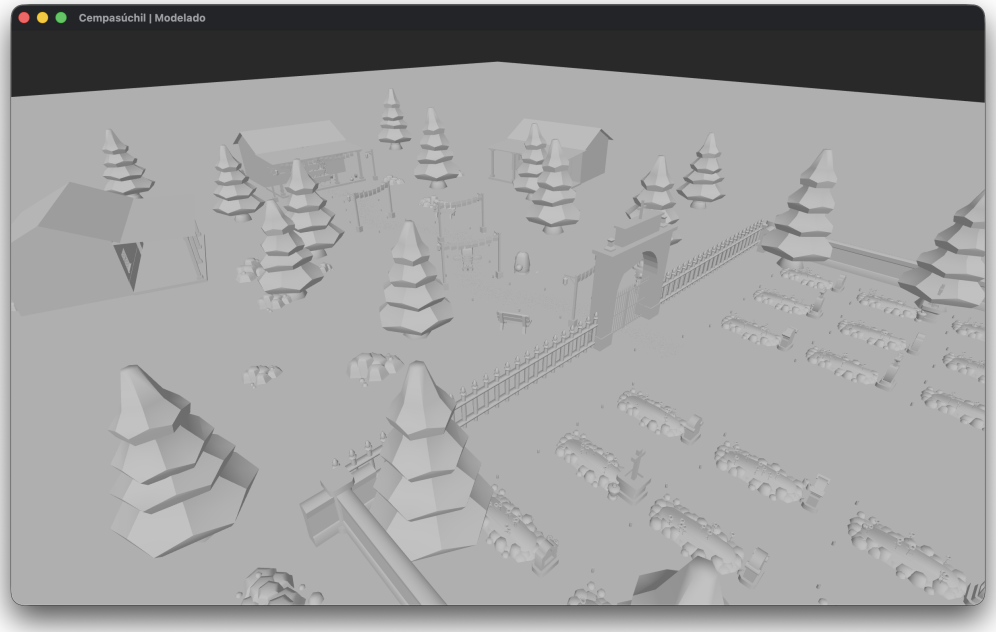


Figura 1: Vista general de la escena.

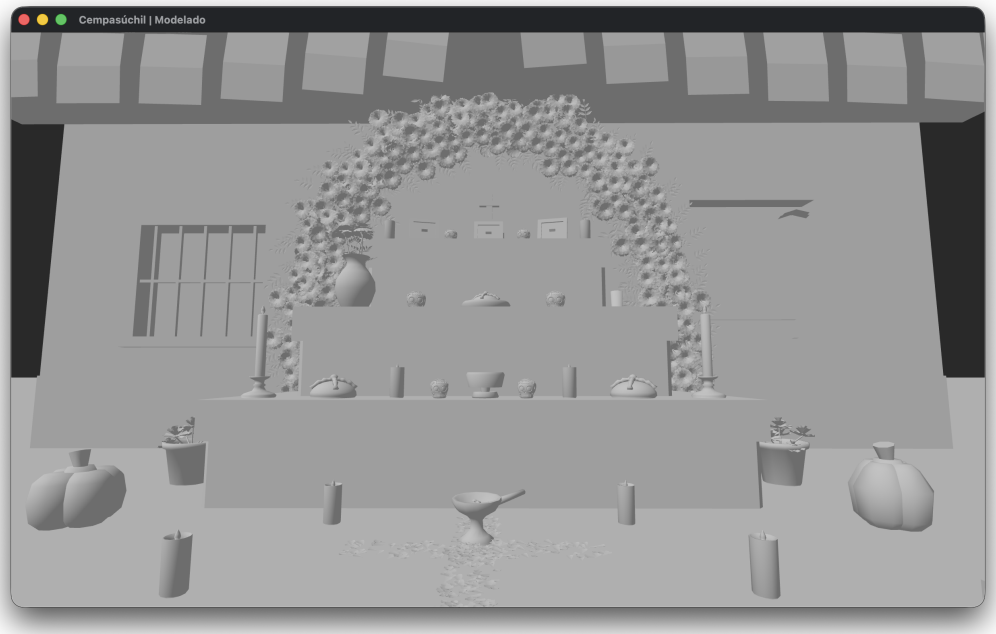


Figura 2: Altar y ofrenda de cerca.



Figura 3: Vista general en modo malla de alambre.

9. Compilación y ejecución

El código es común a los tres sistemas, pero el ejecutable debe compilarse en cada plataforma. CMake lee `CMakeLists.txt` y genera los archivos apropiados.

9.1. macOS con Xcode

```
cmake -S . -B build-xcode -G Xcode
open build-xcode/ModeladoCempasuchil.xcodeproj
```

En Xcode se selecciona `ModeladoCempasuchil`, destino `My Mac`, y se usa `Cmd+R`. `build-xcode` es una carpeta generada y reconstruible.

9.2. Windows con Visual Studio

Se requiere Visual Studio 2022 con “Desarrollo para el escritorio con C++” y CMake.

```
cmake -S . -B build-windows -G "Visual Studio 17 2022" -A x64
cmake --build build-windows --config Release
```

El ejecutable queda en `build-windows/Release`. `assets`, `shaders` y `readme.txt` deben permanecer junto al archivo `.exe`. Posteriormente puede empaquetarse con InstallForge.

9.3. Linux

En una distribución basada en Ubuntu:

```
sudo apt update
sudo apt install build-essential cmake libx11-dev libxrandr-dev “
            libxinerama-dev libxcursor-dev libxi-dev
cmake -S . -B build -DCMAKE˙BUILD˙TYPE=Release
cmake --build build -j
./build/ModeladoCempasuchil
```

Los nombres de paquetes pueden cambiar según la distribución. Se requieren controladores compatibles con OpenGL 3.3 o superior.

9.4. Controles

Acción	Control
Avanzar, retroceder y desplazarse	W, S, A, D
Bajar o subir	Q, E
Mirar	Botón derecho y movimiento del ratón
Acelerar	Shift
Ajustar velocidad	Rueda del ratón
Alternar sólido/alambre	F2
Volver al inicio	R
Salir	Esc

Tabla 4: Controles de la entrega de modelado.

10. Resultados

Se obtuvo una escena navegable formada por una casa con ofrenda, un camino de cempasúchil y un panteón. Los dieciocho modelos del equipo se integraron con objetos de librería para completar el contexto. El acabado uniforme permite comparar formas sin que el color o las texturas oculten errores de proporción.

El modo de alambre demuestra que los objetos están constituidos por mallas trianguladas y permite identificar zonas con mayor densidad. Las instancias hacen posible poblar el arco, el camino y el panteón sin duplicar innecesariamente cada malla en memoria.

11. Conclusiones y trabajo posterior

La entrega establece una base geométrica funcional. La separación entre carga de modelos, escena, cámara y render permite modificar una parte sin reescribir las demás. glTF facilita la comunicación entre Blender y OpenGL, mientras que el grafo jerárquico conserva relaciones entre piezas.

La metodología ágil permite asignar modelos, integrar avances incrementales y detectar errores antes del cierre. Para demostrar su aplicación, el equipo conservará el tablero de tareas, los *commits*, las revisiones y una retrospectiva breve de cada iteración.

En la siguiente etapa se incorporarán materiales, color, texturas e iluminación, respetando la geometría validada. Después podrán añadirse animaciones e interacción.

12. Referencias

- Blender Foundation. (2026). *Blender 5.2*. https://www.blender.org/
- Khronos Group. (2021). *glTF 2.0 Specification*. https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html
- Khronos Group. (2010). *OpenGL 3.3 Core Profile Specification*. https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf
- Kenney. (2025). *Graveyard Kit 5.0* [Modelos 3D, CC0]. https://kenney.nl/assets/graveyard-kit
- Schwaber, K. y Sutherland, J. (2020). *The Scrum Guide*. https://scrumguides.org/